ARQUITETURAS DE COMPUTADORES

TÓPICO 2 – MICROARQUITETURA (AULA 12)

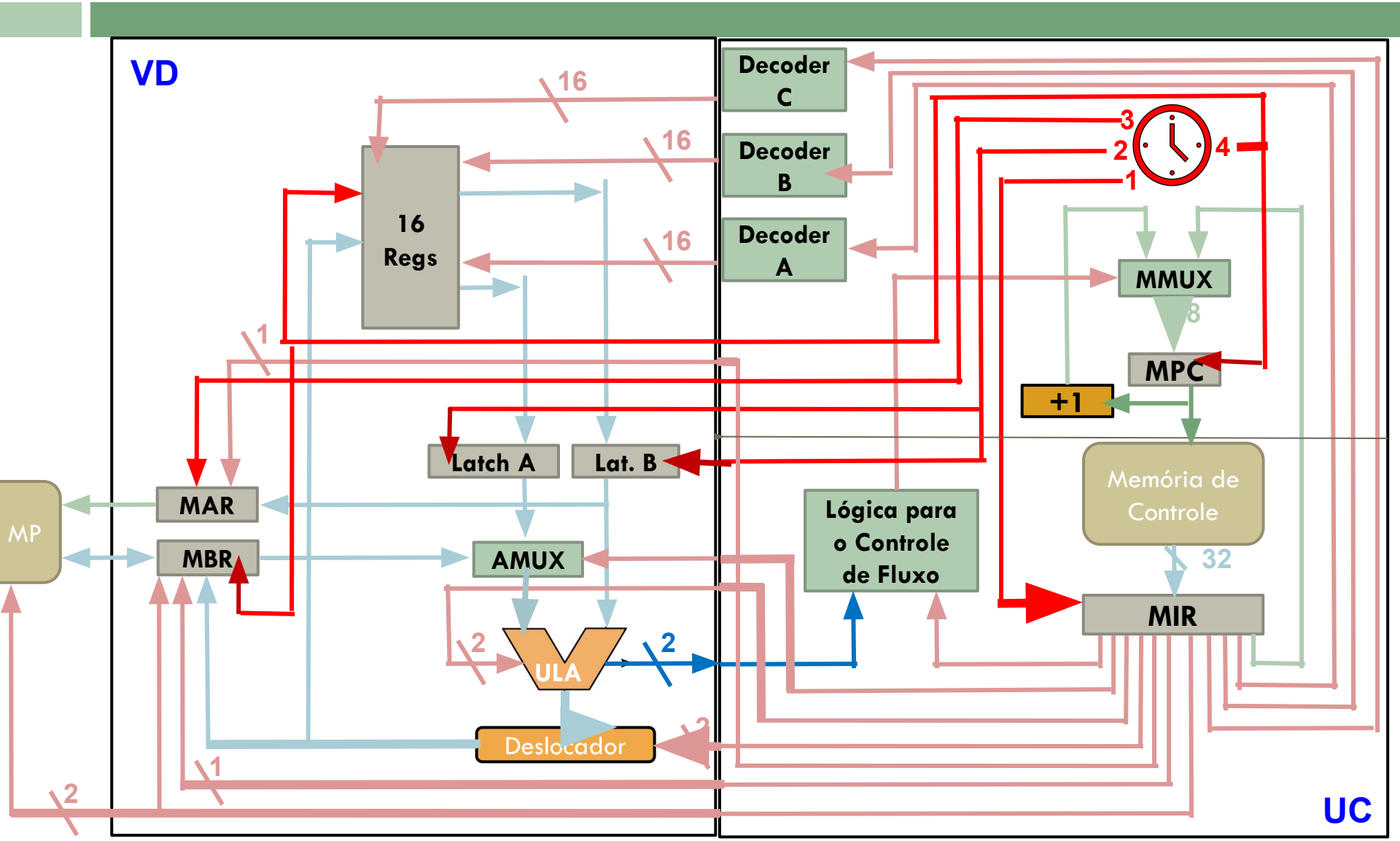
Cristina Boeres, cristina_boeres@id.uff.br

A microinstrução de MIC-1



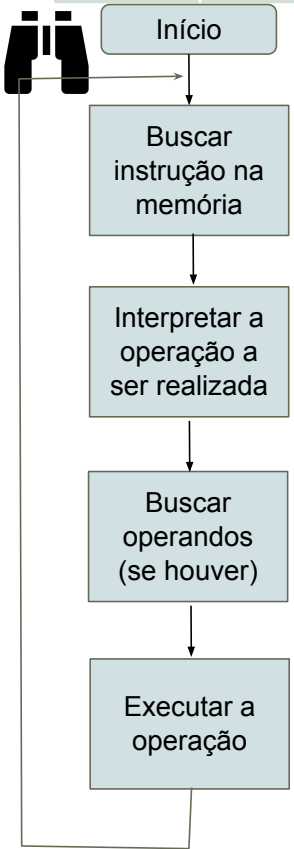
- A (micro)programa é **uma sequência** de (micro)instruções
  - Precisamos realizar **desvios** condicionais e incondicionais:
    - Se COND = 00, não desvia; Se COND = 01 e **N = 1**, desvia; Se COND = 10 e **Z = 1**, desvia; Se COND = 11, desvia para o endereço indicado no ADDR.
    - Os bits de status da ULA: **N** identifica se o resultado da ULA é negativo e **Z** indica que os resulto da ULA é zero
    - **Nesta versão de MIC-1 não tratamos estorno (*overflow*) nos resultados de soma na ULA**
- Com 12 bits de opcode parece que tem 4096 possíveis microinstruções em L1...

O projeto do microprocessador MIC-1



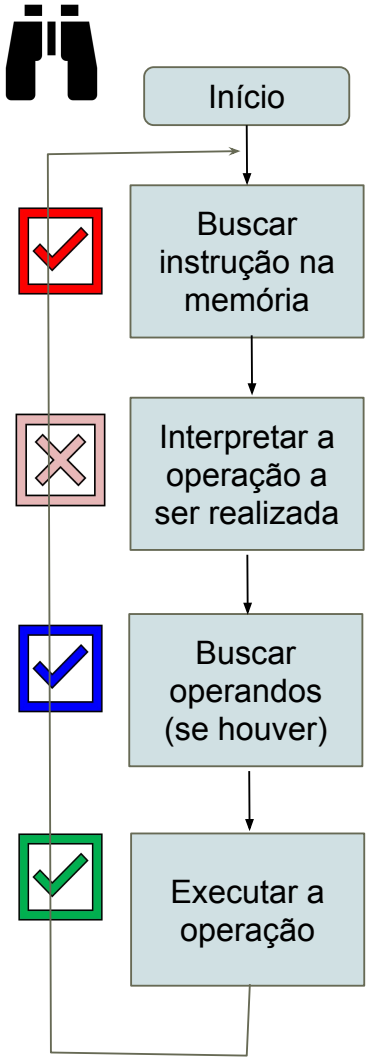
O Ciclo de VN para o Macroprograma

OPCODE									OPERANDOS			
AMux (1)	COND (2)	ULA (2)	DESL (2)	MBR (1)	MAR (1)	RD (1)	WR (1)	En C (1)	Bar. C (4)	Bar. B (4)	Bar. A (4)	ADDR (8)



- O µprograma tem que interpretar o ***macroprograma***, isto é, implementa o modelo VN do macroprograma
- Por ser um programa, o ciclo VN seria realizada em **software**
  - Busca a próxima **macroinstrução**
  - Identifica a macroinstrução buscada
  - Executa uma sequência de ***microinstruções*** equivalente a macroinst
  - Repete

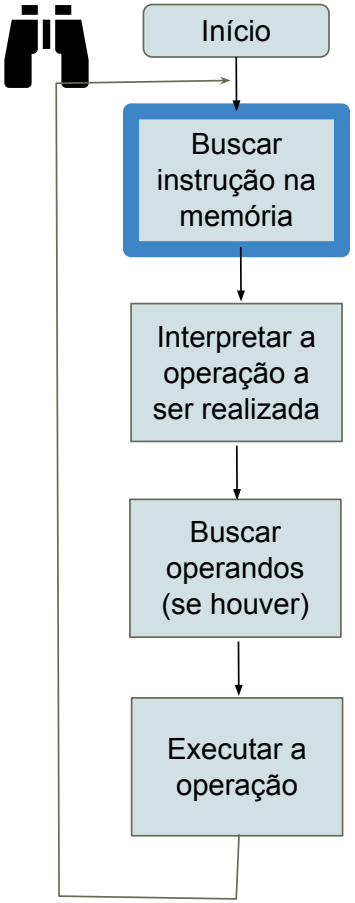
O Microprograma de MIC-1



- O **microprograma** tem que interpretar o macroprograma
  - Busca a próxima macroinstrução
  - Identifica a macroinstrução buscada
  - Executa uma sequência de microinstruções equivalente
  - Repete
- Já vimos como usar o Via de Dados
- A UC determina o que acontece, e quando, durante um ciclo de relógio
- O **microprograma** define que acontece ciclo a ciclo
- A interpretação ou realização de uma macroinstrução leva vários ciclos (requer a execução de várias microinstruções)
- *Por que?*

Busca na memória de MIC-1

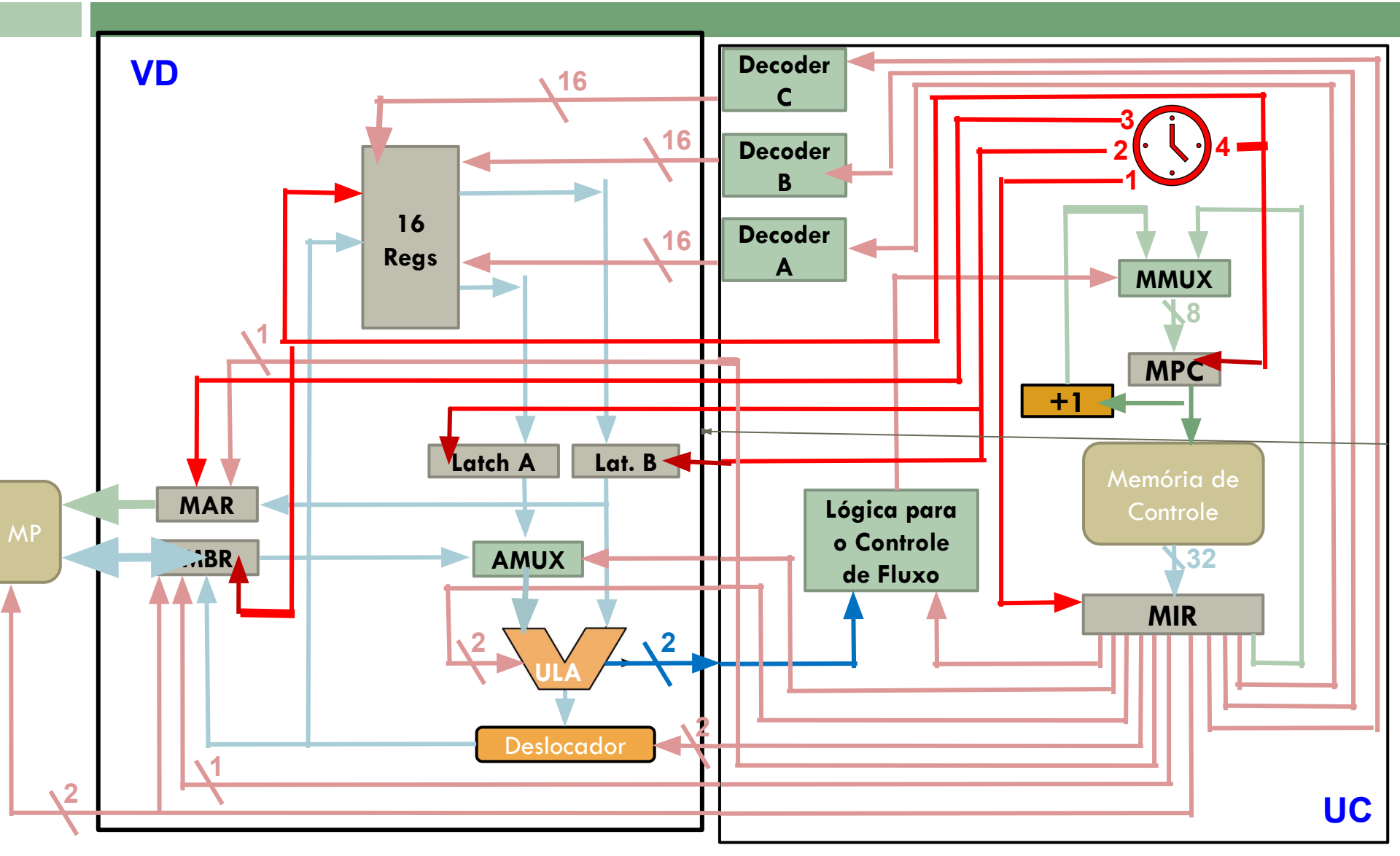
	OPCODE									OPERANDOS			
	AMux (1)	COND (2)	ULA (2)	DESL (2)	MBR (1)	MAR (1)	RD (1)	WR (1)	En C (1)	Bar. C (4)	Bar. B (4)	Bar. A (4)	ADDR (8)
0:	0	00	00	00	0	1	1	0	0	0000	0000	0000	$(01)_{16}$



- Dado que a latência da "memória principal" é seis subciclos
- IR = mp[PC];
  - 0: mar := pc; rd;
  - 1: pc := pc + 1; rd;
  - 2: ir := mbr;

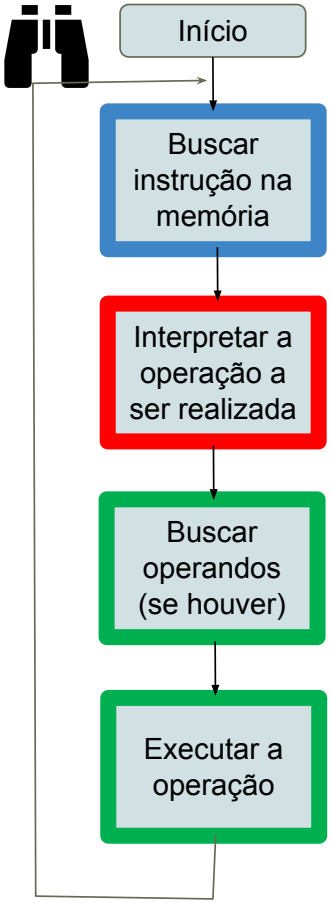
C	SC	Ação
1	1	Busca da μinst
1	2	Busca pc, rd=1
1	3	End = ??, rd=1
1	4	**mar := pc; rd=1**
2	1	Busca da μinst; **rd=1**
2	2	Busca pc,1; **rd=1**
2	3	Soma; **rd=1**
2	4	Soma; **rd=1**
3	1	Busca da μinst; **rd=1**
3	2	Amux=1; EnC=1; **rd=0**

O projeto do microprocessador MIC-1



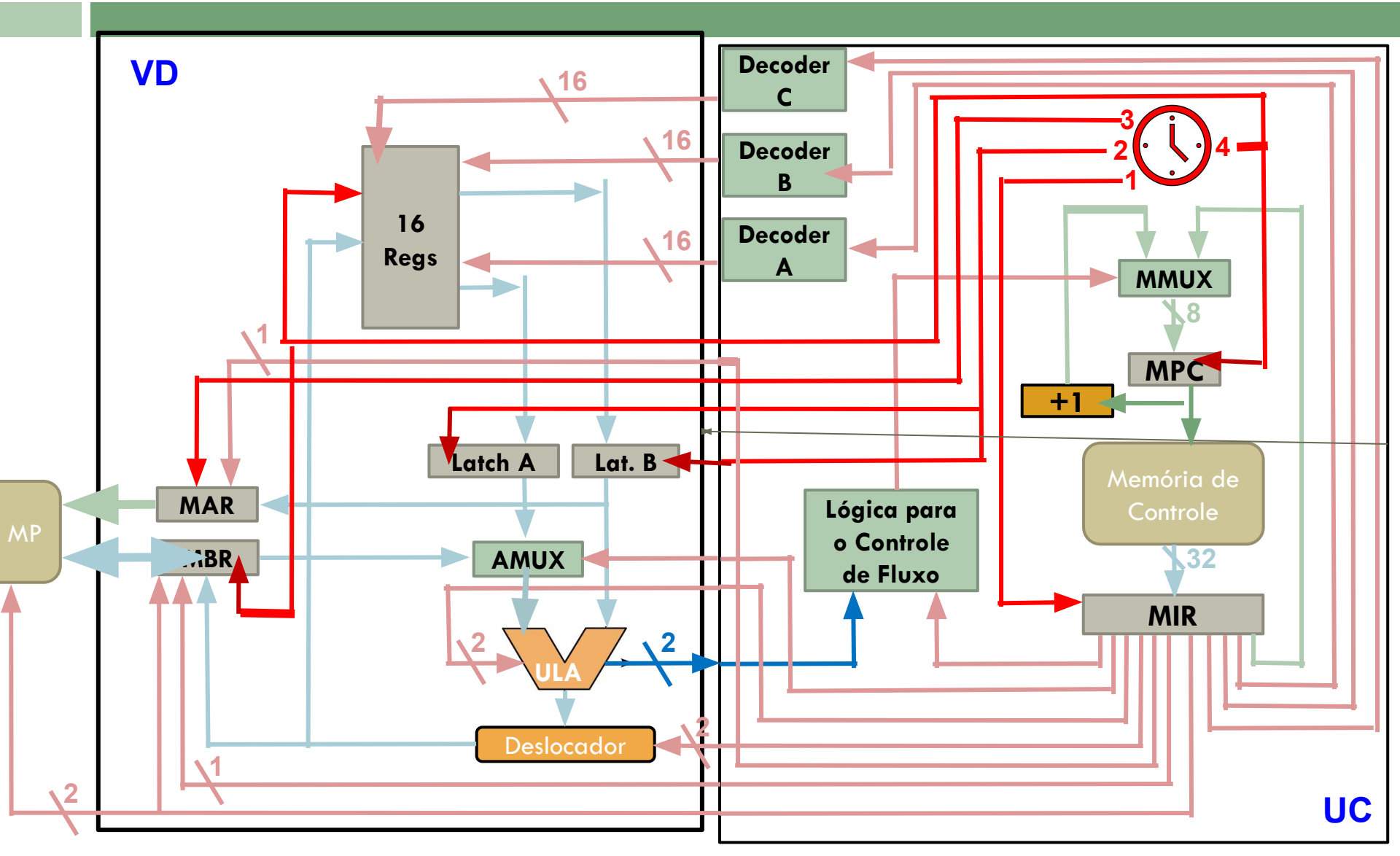
Identificando a macroinstrução

	OPCODE								OPERANDOS			
	AMux (1)	COND (2)	ULA (2)	DESL (2)	MBR (1)	MAR (1)	RD (1)	WR (1)	En C (1)	Bar. C (4)	Bar. B (4)	Bar. A (4)	ADDR (8)
3:	0	01	00	10	0	0	0	0	1	0100	0011	0011	$(13)_{16}$



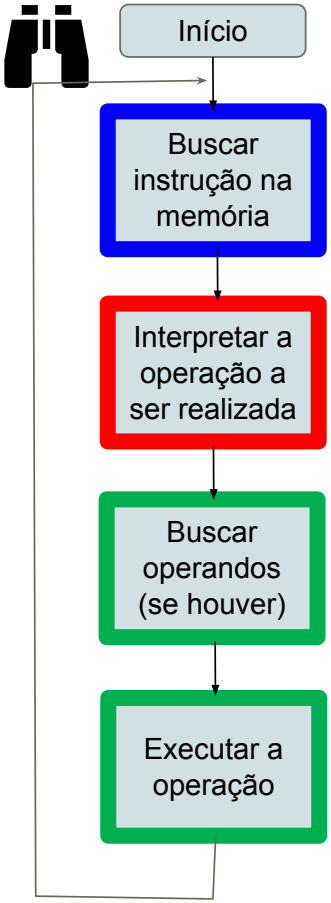
- Macroinstrução tem 16 bits:
  - 2: ir := mbr; if n then goto 28; — Interpreta
  - 3: tir := lshift(ir + ir); if n then goto 19; — A instrução é *negativo*?
  - 4: tir := lshift(tir); if n then goto 11;
  - 5: alu := tir; if n then goto 9;
  - 6: mar := ir; rd; — Executa {0000 = LODD}
  - 7: rd; — Executa
  - 8: ac := mbr; goto 0; — Executa
  - 9: mar := ir; mbr := ac; wr; — Executa {0001 = STOD}
  - 10: wr; goto 0; — Executa

O projeto do microprocessador MIC-1



Identificando a macroinstrução

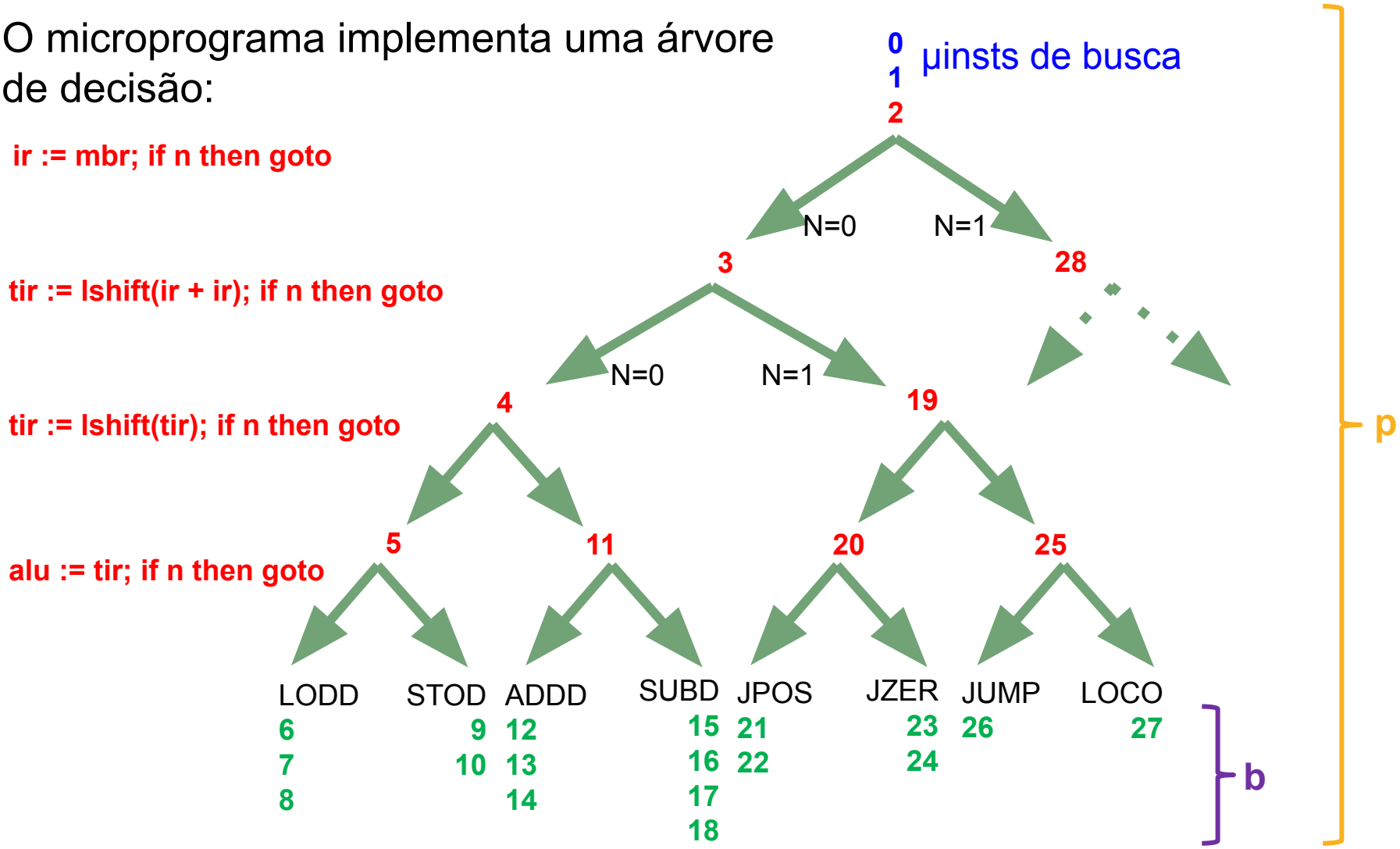
	OPCODE								OPERANDOS			
	AMux (1)	COND (2)	ULA (2)	DESL (2)	MBR (1)	MAR (1)	RD (1)	WR (1)	En C (1)	Bar. C (4)	Bar. B (4)	Bar. A (4)	ADDR (8)
3:	0	01	00	10	0	0	0	0	1	0100	0011	0011	13



- Macroinstrução tem 16 bits: ABCDEFGHIJKLMNOP
  - 2: ir := mbr; if n then goto 28; *if A = 1 then goto 28*
  - 3: tir := lshift(ir + ir); if n then goto 19; *if B = 1 then goto 19*
  - 4: tir := lshift(tir); if n then goto 11; *if C = 1 then goto 11*
  - 5: alu := tir; if n then goto 9; *if D = 1 then goto 9*
  - 6: mar := ir; rd; Executa {0000 = LODD}
  - 7: rd; Executa
  - 8: ac := mbr; goto 0; Executa
  - 9: mar := ir; mbr := ac; wr; Executa {0001 = STOD}
  - 10: wr; goto 0; Executa

Identificando a macroinstrução

- O microprograma implementa uma árvore de decisão:



O Microprograma de MIC-1

Este microprogram tem 79 microinstruções de 32 bits

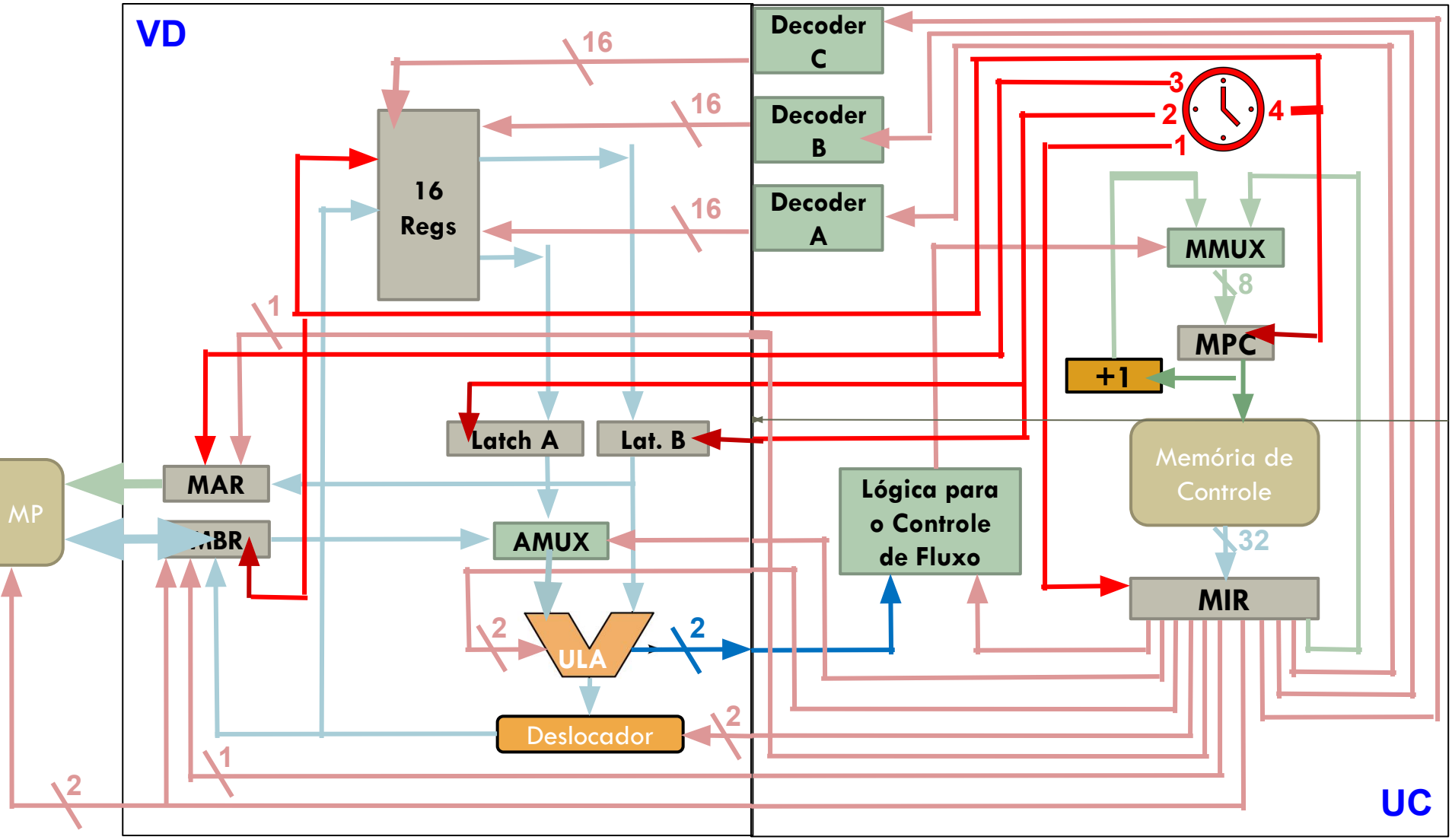
0: mar:=pc; rd;	Busca
1: pc:=pc + 1; rd;	Busca
2: ir:=mbr; if n then goto 28;	Busca/Identifica
3: tir:=lshift(ir + ir); if n then goto 19;	Identifica
4: tir:=lshift(tir); if n then goto 11;	Identifica
5: alu:=tir; if n then goto 9;	Identifica
6: mar:=ir; rd;	Executa {0000 = LODD}
7: rd;	Executa
8: ac:=mbr; goto 0;	Executa
9: mar:=ir; mbr:=ac; wr;	{0001 = STOD}
10: wr; goto 0;	Executa
11: alu:=tir; if n then goto 15;	Identifica
12: mar:=ir; rd;	{0010 = ADDD}
13: rd;	Executa
14: ac:=mbr + ac; goto 0;	Executa
15: mar:=ir; rd;	{0011 = SUBD}
16: ac:=ac + 1; rd;	Executa
17: a:=inv(mbr);	Executa
18: ac:=ac + a; goto 0;	Executa
19: tir:=lshift(tir); if n then goto 25;	Identifica
20: alu:=tir; if n then goto 23;	
21: alu:=ac; if n then goto 0;	{0100 = JPOS}
22: pc:=band(ir,amask); goto 0;	
23: alu:=ac; if z then goto 22;	{0101 = JZER}
24: goto 0;	
25: alu:=tir; if n then goto 27;	
26: pc:=band(ir,amask); goto 0;	{0110 = JUMP}
27: ac:=band(ir,amask); goto 0;	{0111 = LOCO}
28: tir:=lshift(ir + ir); if n then goto 40;	
29: tir:=lshift(tir); if n then goto 35;	
30: alu:=tir; if n then goto 33;	
31: a:=ir + sp;	{1000 = LODL}
32: mar:=a; rd; goto 7;	
33: a:=ir + sp;	{1001 = STOL}
34: mar:=a; mbr:=ac; wr; goto 10;	
35: alu:=tir; if n then goto 38;	
36: a:=ir + sp;	{1010 = ADDL}
37: mar:=a; rd; goto 13;	
38: a:=ir + sp;	{1011 = SUBL}
39: mar:=a; rd; goto 16 ;	

40: tir:=lshift(tir); if n then goto 46;	
41: alu:=tir; if n then goto 44;	
42: alu:=ac; if n then goto 22;	{1100 = JNEG}
43: goto 0;	
44: alu:=ac; if z then goto 0;	{1101 = JNZE}
45: pc:=band(ir,amask); goto 0;	
46: tir:=lshift(tir); if n then goto 50;	
47: sp:=sp + (-1);	{1110 = CALL}
48: mar:=sp; mbr:=pc; wr;	
49: pc:=band(ir,amask); wr; goto 0;	
50: tir:=lshift(tir); if n then goto 65;	
51: tir:=lshift(tir); if n then goto 59;	
52: alu:=tir; if n then goto 56;	
53: mar:=ac; rd;	{1111-0000 = PSHI}
54: sp:=sp + (-1); rd;	
55: mar:=sp; wr; goto 10;	
56: mar:=sp; sp:=sp + 1; rd;	{1111-0010 = POPI}
57: rd;	
58: mar:=ac; wr; goto 10;	
59: alu:=tir; if n then goto 62;	
60: sp:=sp + (-1);	{1111-0100 = PUSH}
61: mar:=sp; mbr:=ac; wr; goto 10;	
62: mar:=sp; sp:=sp + 1; rd;	{1111-0110 = POP}
63: rd;	
64: ac:=mbr; goto 0;	
65: tir:=lshift(tir); if n then goto 73;	
66: alu:=tir; if n then goto 70;	
67: mar:=sp; sp:=sp + 1; rd;	{1111-1000 = RETN}
68: rd;	
69: pc:=mbr; goto 0;	
70: a:=ac;	{1111-1010 = SWAP}
71: ac:=sp;	
72: sp:=a; goto 0;	
73: alu:=tir; if n then goto 76;	
74: a:=band(ir,smask);	{1111-1100 = INSP}
75: sp:=sp + a; goto 0;	
76: a:=band(ir, smask);	{1111-1110 = DESP}
77: a:=inv(a);	
78: a:=a + 1; goto 75;	

O Microprograma de MIC-1

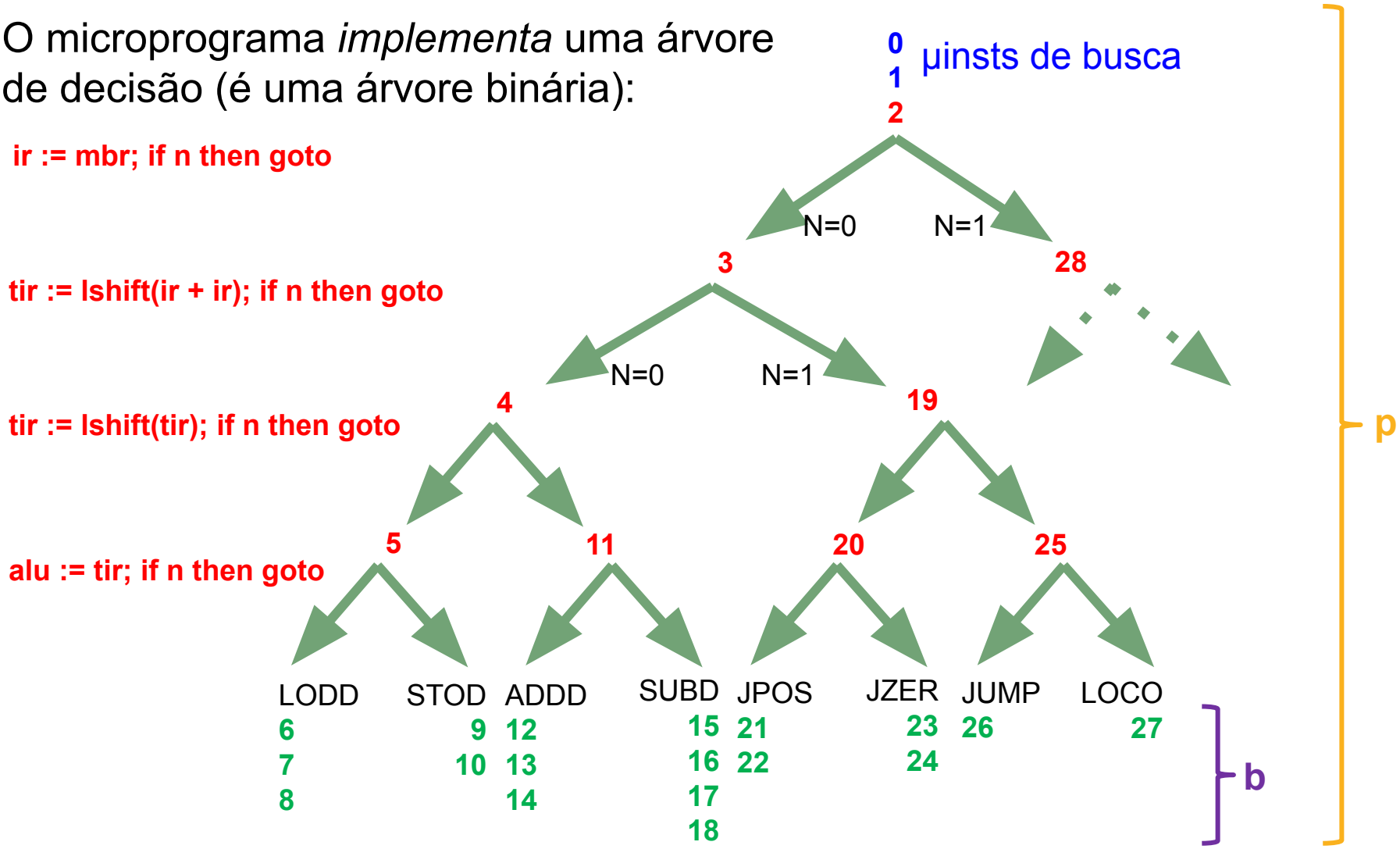
- A memória de controle tem 256 × 32 bits, isto é 8Ki bits,
  - o microprograma ocupa só 79 × 32 = 2528 bits
  - Mas não é a única implementação
  - Têm possibilidades para reduzir o tamanho da microinstrução sem perder funcionalidade ...
- Na *média* 9 ciclos (μinsts) para executar uma macroinstrução
- Identificar cada macroinstrução tem um *custo* significativo
  - Será que tem uma maneira para reduzir este custo?
    - Em hardware, é muito provável. Como programador, você faria o que?
- MIC-1 (ainda) não é uma implementação otimizada em termos de desempenho e custo

O projeto do microprocessador MIC-1



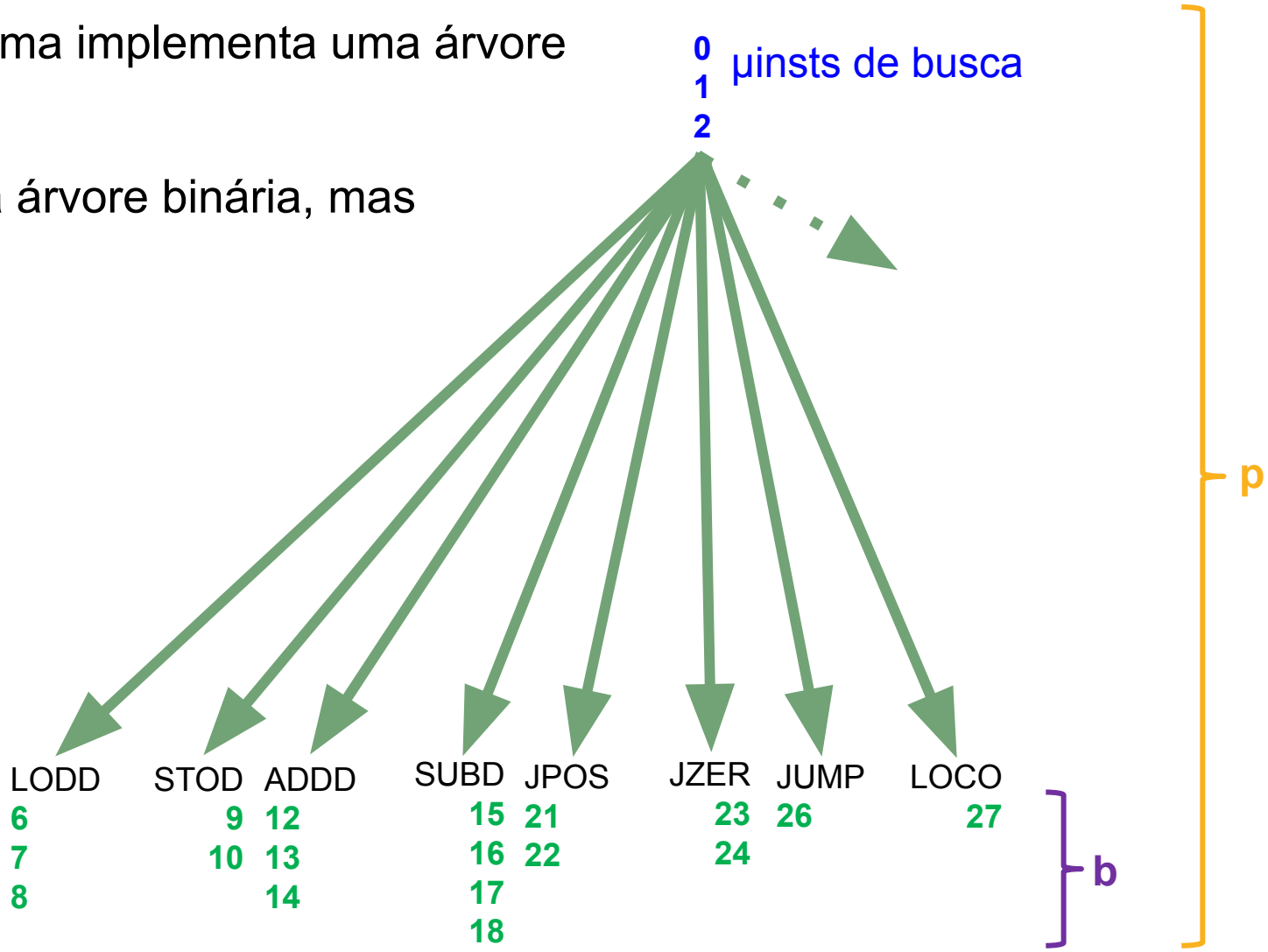
O Custo de Interpretação

- O microprograma *implementa* uma árvore de decisão (é uma árvore binária):

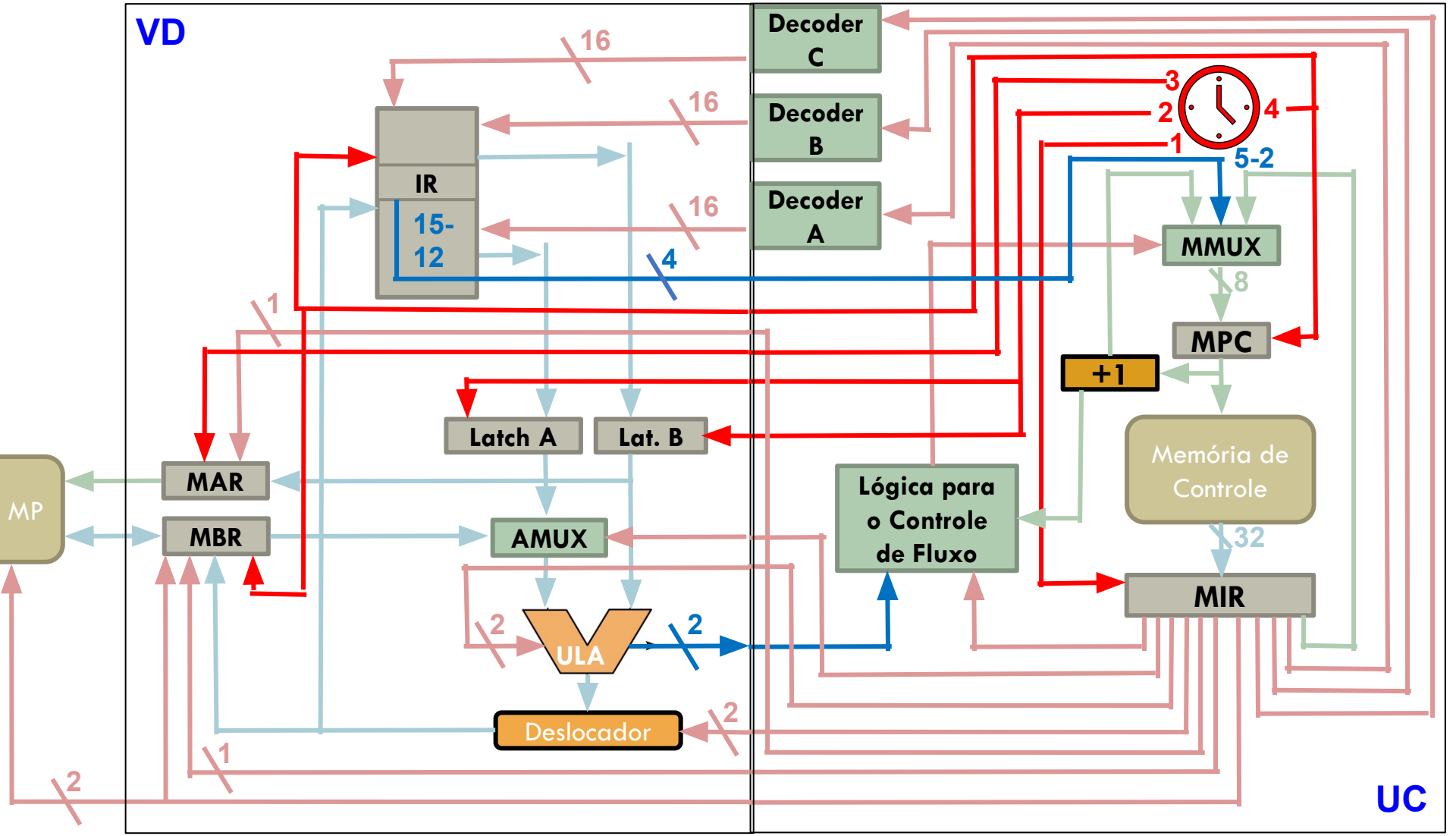


Identificação usando um "*Case*"?

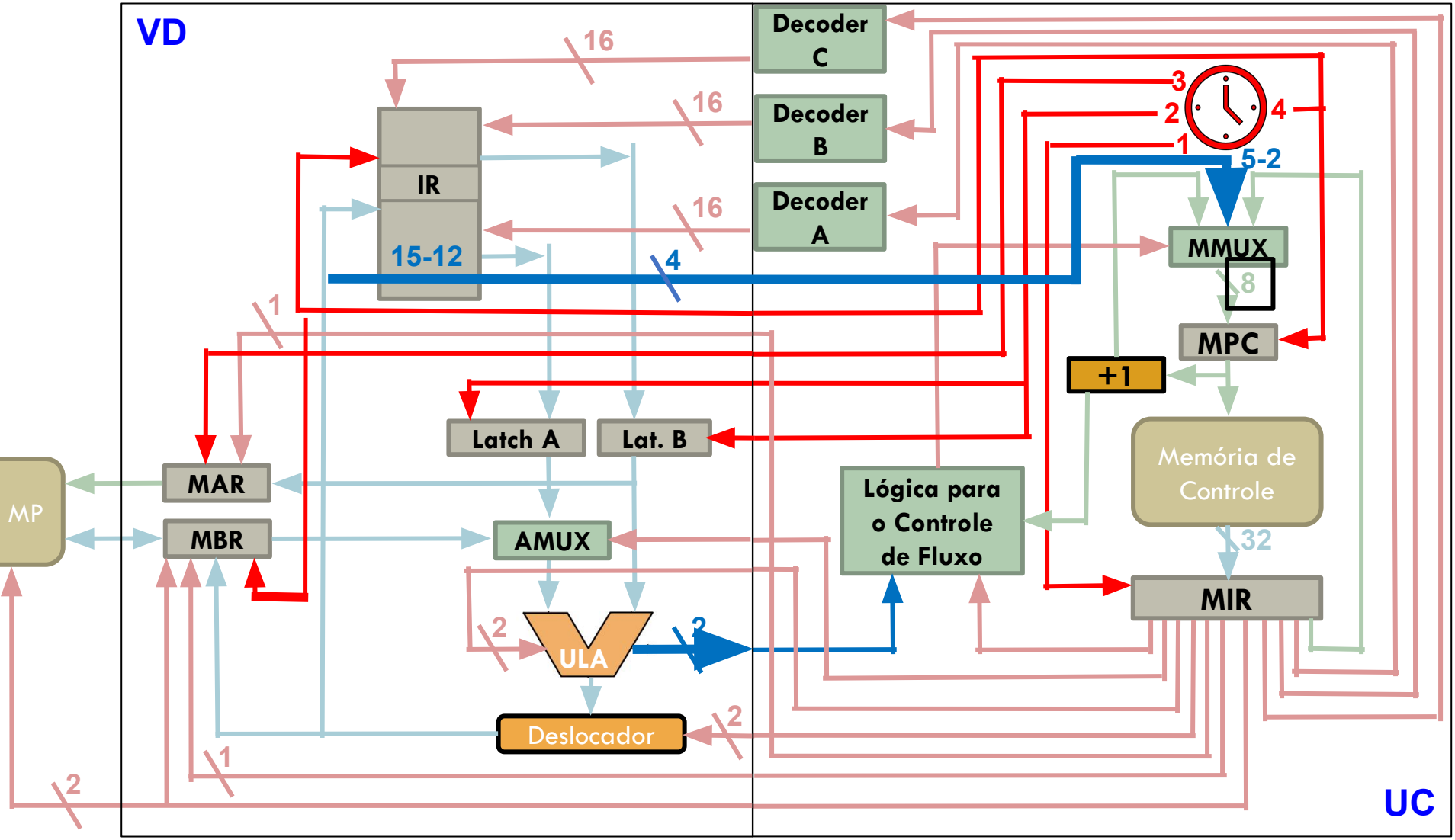
- O microprograma implementa uma árvore de decisão:
- Não seria uma árvore binária, mas *multi-way*



Implementação do "*Case*" no MIC-1



Implementação do "*Case*" no MIC-1

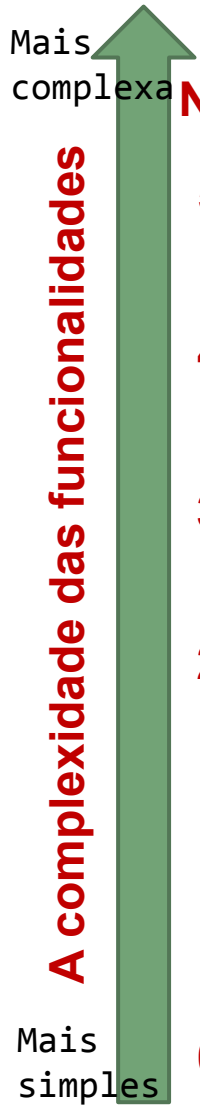


Identificando a macroinstrução

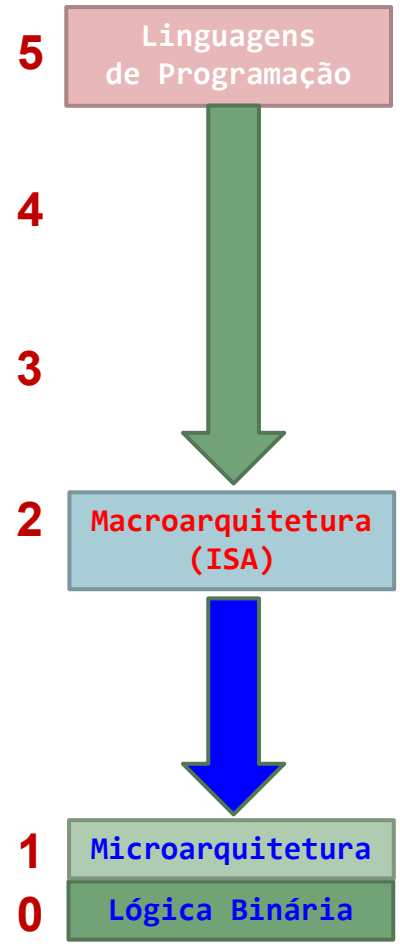
- Microprograma original:
  - 0: mar := pc; rd;
  - 1: pc := pc + 1; rd;
  - 2: ir := mbr; if n then goto 28;
  - 3: tir := lshift(ir + ir); if n then goto 19;
  - 4: tir := lshift(tir); if n then goto 11;
  - 5: alu := tir; if n then goto 9;
  - 6: mar := ir; rd;
  - 7: rd;
  - 8: ac := mbr; goto 0;
  - 9: mar := ir; mbr := ac; wr;
  - 10: wr; goto 0;
  - 11: ...

- Microprograma com suporte h'ware:
  - 0: mar := ir; rd; ['0000' = LODD]
  - 1: rd;
  - 2: ac := mbr; goto 252;
  - 3:
  - 4: mar := ir; mbr := ac; wr;
  - 5: wr; goto 252; ['0001' = STOD]
  - 6:
  - 60: tir := lshift(ir+ir);
  - 61: tir := lshift(tir+tir);
  - 62: tir := lshift(tir); if n then goto 77;
  - 63: ... 90: µinsts originais 51 a 78
  - 252: mar := pc; rd;
  - 253: pc := pc + 1; rd;
  - 254: ir := mbr; [mpc = 255]

Tópico 2: A grande pergunta

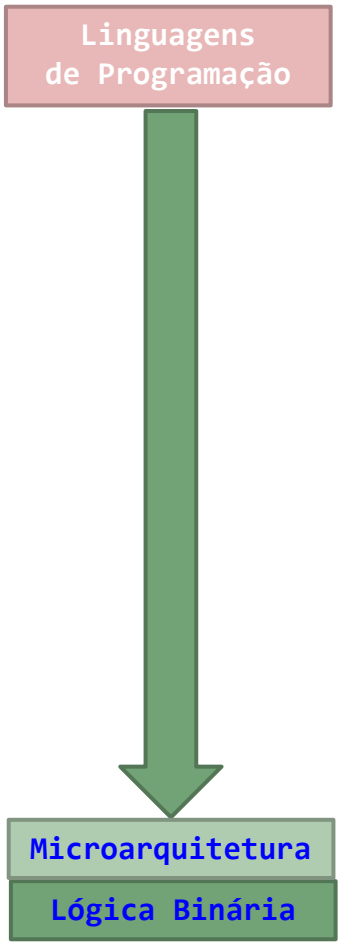


Qual abordagem executa programas mais rápido?



Opção A: Tradução (Compilador + Montador) e Interpretação (no Processador)
Fazendo interpretação não está atrasando a execução?

Opção B: Só Tradução, (Com um Compilador gerando instruções de nível 1 diretamente)
Quão fácil será para o compilador fazer isso?
Haverá um impacto adverso no desempenho?



Se ($i \leq n$) então $m := a[i]$;

Sendo um compilador:

- (Opção A) Traduz diretamente para uma sequência de ***macroinstruções*** correspondente
  - Qual é o desempenho e custo do macroprograma?

Nossa ISA

ISA MAC-1 (nível 2)

- 23 Macroinstruções de 16 bits (coluna 1)
  - **Opcode** crescente (4, 8 ou 16 bits), tamanho não fixo
  - Tipos de endereçamento:
    - Endereçamento direto
    - Endereçamento indexado
    - Sem operando
  - Um operando de 12 bits ou 8 bits ou sem operando
    - Note que, ter um operando na instrução, não determina o número de dados necessários para a respectiva operação

Encoding	Mnemonic	Instruction	Semantics
0000xxxxxxxxxxxx	LODD	Load Direct	ac := m[x]
0001xxxxxxxxxxxx	STOD	Store Direct	m[x] := ac
0010xxxxxxxxxxxx	ADDD	Add Direct	ac := ac + m[x]
0011xxxxxxxxxxxx	SUBD	Subtract Direct	ac := ac - m[x]
0100xxxxxxxxxxxx	JPOS	Jump on non-negative	if ac ≥ 0, pc := x
0101xxxxxxxxxxxx	JZER	Jump on zero	if ac = 0, pc := x
0110xxxxxxxxxxxx	JUMP	Jump	pc := x
0111xxxxxxxxxxxx	LOCO	Load Constant	ac := x ((0 ≤ x ≤ 4095)
1000xxxxxxxxxxxx	LODL	Load Local	ac := m[sp + x]
1001xxxxxxxxxxxx	STOL	Store Local	m[sp + x] := ac
1010xxxxxxxxxxxx	ADDL	Add Local	ac := ac + m[sp + x]
1011xxxxxxxxxxxx	SUBL	Subtract Local	ac := ac - m[sp + x]
1100xxxxxxxxxxxx	JNEG	Jump on negative	if ac < 0, pc := x
1101xxxxxxxxxxxx	JNZE	Jump unless zero	if ac ≠ 0, pc := x
1110xxxxxxxxxxxx	CALL	Call	sp := sp - 1; m[sp] := pc; pc := x
1111000000000000	PSHI	Push Indirect	sp := sp - 1; m[sp] := m[ac]
1111001000000000	POPI	Pop Indirect	m[ac] := m[sp]; sp := sp + 1
1111010000000000	PUSH	Push	sp := sp - 1; m[sp] := ac
1111011000000000	POP	Pop	ac := m[sp]; sp := sp + 1
1111100000000000	RETN	Return	pc := m[sp]; sp := sp + 1
1111101000000000	SWAP	Swap AC , SP	tmp:= ac;ac :=: sp;sp=tmp
11111100yyyyyyyy	INSP	Increment SP	sp := sp + y (0 ≤ y ≤ 255)
11111110yyyyyyyy	DESP	Decrement SP	sp := sp - y (0 ≤ y ≤ 255)

xxxxxxxxxxxx is a 12-bit machine address; in column 4, it is called x.
yyyyyyyy is a 8-bit constant; in column 4 it is called y.

Se ($i \leq n$) então $m := a[i]$;

- Traduz diretamente para uma sequência de ***macroinstruções*** correspondentes:

- Custo: 8 × 16 bits
- Desempenho:
  - (3 ou 8) × ~9 ciclos
    - 26 ou 27 + 47 ciclos

```
        LODD  &n      |  LODD  &i
        SUBD  &i      |  SUBD  &n
        JNEG  saída   |  JZER  dentro
                      |  JPOS  saída
dentro: LOCO  &a
        ADDD  &i
        PSHI
        POP
        STOD  &m
saída:  ...
```

Se ($i \leq n$) então $m := a[i]$;

- Traduz diretamente para uma sequência de ***macroinstruções*** correspondentes:

- Custo: 8 × 16 bits
- Desempenho:
  - (3 ou 8) × ~9 ciclos
    - 26 ou 27 + 47 ciclos

```
        LODD  &n
        SUBD  &i
        JNEG  saída
```

```
        LODD  &i
        SUBD  &n
        JZER  dentro
        JPOS  saída
```

✗

```
dentro: LOCO  &a
        ADDD  &i
        PSHI
        POP
        STOD  &m
saída:  ...
```

Se ($i \leq n$) então $m := a[i]$;

Sendo um novo compilador:

- (Opção B) Traduz diretamente para um conjunto de ***microinstruções*** correspondentes
  - Qual é o desempenho e custo do microprograma?

Se ($i \leq n$) então $m := a[i]$;

Sendo um novo compilador:

- (Opção B) Traduz diretamente para um conjunto de ***microinstruções*** correspondentes
  - Qual é o desempenho e custo do microprograma?
- Este é um exercício puramente hipotético
  - A arquitetura MIC-1, como está, não suporta este estilo de execução
  - Este exercício é só para validar a hipótese de Von Neumann

Se ($i \leq n$) então $m := a[i]$;

- Traduz diretamente para uma sequência de microinstruções correspondentes:
- **Desempenho**:
  - 7 ou 14 ciclos **se usa a MC**
    - 5 vezes melhor 
- **Custo**: 14 × 32 bits
  - ~4 vezes mais p/ 1 linha 
  - Para um programa(s) inteiro?
    - μprograma só custa 316 bytes

```
mar := &n; rd       # busca n
rd;                 # 2cicl p/ler
ac := mbr;          # ac recebe n
mar := &i; rd       # busca i
rd; ac := ac + 1    # subtração 1
a := inv(mbr);      # subtração 2
alu := ac + a; if n goto saída;
rd;                  # MAR já tem &i
ac := &a; rd;   # carreg. end. a
ac := mbr + ac;     # end. a[i]
mar := ac; rd;      # busca a[i]
rd;
ac := mbr;
mar := &m; mbr := ac; wr;
wr;                 # atualizar m
saída: ...
```

Se ($i \leq n$) então $m := a[i]$;

- Traduz diretamente para uma sequência de microinstruções correspondentes:
- **Desempenho**:
  - 7 ou 14 ciclos **se usa a MC**
    - 5 vezes melhor 
  - ~4 vezes mais p/ 1 linha 
  - Para um programa(s) inteiro?
    - μprograma só custa 316 bytes
- **Custo**: 14 × 32 bits



```
mar := &n; rd       # busca n
rd;                 # 2cicl p/ler
ac := mbr;          # ac recebe n
mar := &i; rd       # busca i
rd; ac := ac + 1    # subtração 1
a := inv(mbr);      # subtração 2
alu := ac + a; if n goto saída;
rd;                  # MAR já tem &i
ac := &a; rd;   # carreg. end. a
ac := mbr + ac;     # end. a[i]
mar := ac; rd;      # busca a[i]
rd;
ac := mbr;
mar := &m; mbr := ac; wr;
wr;                 # atualizar m
saída: ...
```